CSE440 Section-1 Project Group-8

Bayes Watch: A Bayesian Network-Based Mystery Solver Application for the Case of the Vanishing Sapphire

Md. Nazibul Islam Nabil
2222456042
Department of ECE
North South University
Dhaka, Bangladesh
nazibul.nabil@northsouth.edu

Tasnif Gaffar Pronoy
2222590042
Department of ECE
North South University
Dhaka, Bangladesh
tasnif.pronoy@northsouth.edu

Nazifa Tahsin
2132652642
Department of ECE
North South University
Dhaka, Bangladesh
nazifa.tahsin@northsouth.edu

Zinat Shaharin Mim
2111301042
Department of ECE
North South University
Dhaka, Bangladesh
zinat.mim@northsouth.edu

Jannat-a-habib-baishakhi
2131069642
Department of ECE
North South University
Dhaka, Bangladesh
jannat.baishakhi@northsouth.edu

***Abstract*—**Reasoning under uncertainty is central to any task that must combine several imperfect pieces of evidence into a single, defensible conclusion. This paper presents Bayes Watch, an interactive application that demonstrates probabilistic reasoning through a fictional jewel-theft scenario, "The Case of the Vanishing Sapphire." The system models the relationship between a hidden culprit and eight observable clues — forced entry, motive, alibi, fingerprint, and security-footage evidence — using a Discrete Bayesian Network (DBN) built with the pgmpy library. The network follows a Naive-Bayes-style "one cause, many effects" topology, with the GuiltyParty node as the sole parent of every evidence node, and Conditional Probability Distributions (CPDs) that encode the narrative logic of the case. A Streamlit front end lets the player select the state of each clue — or leave it as "Unknown" — and, on demand, the application runs pgmpy's Variable Elimination algorithm to compute the exact posterior probability distribution P(GuiltyParty | Evidence) over the three suspects. The interface additionally renders the network structure with Graphviz and exposes the raw conditional probability tables, turning the inference engine from a black box into a transparent teaching tool. Bayes Watch demonstrates, in an engaging and accessible form, how Bayesian belief updating integrates disparate evidence into a single, quantitative judgment.

***Index Terms*—**Bayesian networks, probabilistic inference, reasoning under uncertainty, conditional probability, variable elimination, Naive Bayes, decision support systems, Streamlit.

I. INTRODUCTION

Reasoning under uncertainty is fundamental to artificial intelligence and decision-making, particularly when disparate and sometimes conflicting evidence must be synthesized into a single belief [1]. Deterministic, rule-based approaches struggle in exactly this setting because they offer no principled way to combine partial or contradictory clues. Bayesian Networks (BNs) address this gap by providing a graphical framework for representing probabilistic dependencies and for updating beliefs, via Bayes' theorem, as new evidence arrives [2], [3].

This paper introduces Bayes Watch, an interactive application that uses a Discrete Bayesian Network to solve a fictional jewel-theft mystery, "The Case of the Vanishing Sapphire." The probabilistic model is implemented with the pgmpy library [4], while the user-facing interface is built with Streamlit [5]. The application models a hidden GuiltyParty variable together with eight observable evidence variables spanning forced entry, individual motive, individual alibi, fingerprint matches, and security-camera footage. The network's structure encodes the causal assumption GuiltyParty → Evidence, and each edge is quantified by a Conditional Probability Distribution (CPD) authored from the internal logic of the case file. Players enter the clues they have gathered through a sidebar of selectors and, on request, the system runs the Variable Elimination algorithm [6], [7] to compute the posterior distribution P(GuiltyParty | Evidence), revealing how suspicion should shift toward each of the three suspects. Graphviz [8] is used to render the network topology so that players can see, and not merely trust, the reasoning structure behind the verdict.

Bayes Watch is intended as a pedagogical instrument: a game-like wrapper around a rigorous probabilistic core. The

remainder of this paper is organized as follows. Section II reviews relevant literature on Bayesian networks and the software tools used. Section III details the methodology, including the mathematical foundation, the design of the network, and the interface. Section IV reports representative results. Section V discusses the strengths and limitations of the system, Section VI outlines future work, and Section VII concludes the paper.

II. LITERATURE REVIEW

A. Probabilistic Reasoning and Bayesian Networks

Handling uncertainty has been a central concern of artificial intelligence since its earliest logic-based systems proved brittle in the face of incomplete information [1]. Bayesian Networks, formalized in the seminal work associated with conferences such as Uncertainty in Artificial Intelligence (UAI) and the AAAI [2], provide a Directed Acyclic Graph (DAG) representation in which nodes are random variables and edges denote direct probabilistic influence, with Conditional Probability Distributions quantifying each dependency [3]. This structure allows the full joint distribution over all variables to be represented compactly by exploiting conditional independence [2], [3]. Pearl's foundational treatment of belief networks established BNs as the standard framework for evidential reasoning [2], and the approach has since been applied broadly, from medical diagnosis support systems discussed in venues such as AMIA and AIME [9] to fault diagnosis and monitoring in engineering systems [10]. Across these domains, the recurring strength of BNs is their ability to integrate several uncertain sources of evidence into one coherent, quantitative assessment — precisely the capability Bayes Watch demonstrates in a detective-story setting.

B. Software Tools

Bayes Watch's implementation rests on three specialized libraries. pgmpy, introduced at the Scientific Computing with Python (SciPy) conference [4], supplies the classes used to define the network structure, attach TabularCPD objects to each node, and run exact inference through its VariableElimination class. Streamlit [5] provides the reactive, browser-based interface that collects clue selections and re-renders results the moment the model is queried, without requiring any separate front-end framework. Finally, Graphviz [8], a graph-visualization toolkit whose foundational design was presented in the software-engineering literature, is used to draw the network's DAG so that the causal structure behind the numbers is never hidden from the user. Together these tools let Bayes Watch pair a mathematically exact reasoning engine with an approachable, game-like presentation layer.

III. METHODOLOGY

A. Overview of Bayesian Networks and Probabilistic Inference

A Bayesian network is a probabilistic Directed Acyclic Graph (DAG) model [2], [3]. Nodes represent random variables, arcs encode direct dependency, and each node carries a Conditional Probability Distribution (CPD) that quantifies its dependence on its parents. For random variables $X_1, X_2, \ldots, X_n$ and a graph $G = (V, E)$, the joint distribution factorizes as

$$P(X_1, X_2, \ldots, X_n) = \prod_{j=1}^{n} P(X_j \mid Pa(X_j))$$

where $Pa(X_j)$ denotes the parent set of X_j in G. This factorization follows from the assumption that every variable is conditionally independent of its non-descendants given its parents [2], and it is what allows a joint distribution over many variables to be stored as a handful of small, local tables rather than one exponentially large table.

Given a query set $Q \subseteq V$ and an evidence set $E \subseteq V$ observed to take values e, inference asks for the posterior $P(Q \mid E = e)$, computed from the definition of conditional probability as

$$P(Q = q \mid E = e) = P(Q = q, E = e) / P(E = e)$$

where both the numerator and the normalizing denominator are obtained by marginalizing the full joint distribution of Eq. (1) over the remaining, unobserved variables. Algorithms such as Variable Elimination [6], [7] carry out this marginalization efficiently by exploiting the conditional independencies encoded in G, rather than materializing the full joint table.

B. Scenario Design and Knowledge Engineering

Bayes Watch does not learn its parameters from an empirical dataset. Instead, the network's structure and probability tables were hand-authored from the internal logic of a fictional case file, “The Case of the Vanishing Sapphire” (Section III-D). This narrative served as the surrogate for data: it fixed the set of suspects and clue types, and it guided the knowledge-engineering process used to assign every prior and conditional probability in the model (Section III-F). This mirrors a common practice in early-stage decision-support modeling, where domain expertise substitutes for large historical datasets that do not yet exist [10].

C. System Architecture

The application is organized into two cooperating layers. The probabilistic core lives in a backend module, supports/mystery_solver.py, which uses pgmpy to build the DiscreteBayesianNetwork, attach every TabularCPD, and expose helper functions for Graphviz rendering and for converting a CPD into a display-ready pandas DataFrame. The interface layer, main.py, is a Streamlit script that is deliberately kept free of any probabilistic logic: it imports

build_bayesian_network(), create_graphviz_plot(), and cpd_to_dataframe() from the backend, and is otherwise concerned only with page layout, widget state, and presentation. This separation means the entire case — suspect names, clue wording, even the CPD values — can be re-themed without touching the reasoning engine, and conversely the inference logic can be extended without disturbing the interface.

Development was divided into five cooperating modules within main.py, each contributed by a different member of the team: the page header and theming constants (title, suspect names, and colour palette); the model-construction and inference-engine bootstrap that calls build_bayesian_network() and wraps it in a VariableElimination object; the sidebar case file and the eight evidence-input selectors, including the session-state logic behind the "Reset All Clues" button; the query execution path that is triggered by the "Solve the Mystery" button, builds the evidence dictionary, and renders the posterior table, bar chart, and verdict banner; and the default "no evidence yet" view that queries and displays the raw prior distribution so the results panel is never left empty before the player has entered any clues.

D. Fictional Mystery Scenario Description

The concrete narrative used to ground the model is "The Case of the Vanishing Sapphire." On the night of the Ravenswood Masquerade Gala, the legendary Star of Midnight sapphire disappears from the manor's private vault while the doors are locked and the guests are dancing. Three suspects are considered:

- Suspect A — Lady Odalys Voss, "The Heiress." A distant cousin locked in a decades-long feud over the Voss family jewels, present as guest of honor with legitimate vault access.
- Suspect B — Jenkins, "The Butler." Thirty years of loyal service and the only staff member holding a master key to the vault; claims to have heard nothing all night.
- Suspect C — The Velvet Fox, "The Jewel Thief." A never-caught professional rumored to be working the season's high-society events; deliberately given no formal Motive node, since for a professional the jewels themselves are motive enough.

Eight evidence variables capture the clues a player can gather: whether the vault was forced open (ForcedEntry), whether the Heiress or the Butler had a strong motive (MotiveA, MotiveB), whether each of the three suspects has a verifiable alibi (AlibiA, AlibiB, AlibiC), whose fingerprints, if any, were recovered from the scene (Fingerprints, with states None/A/B/C), and whether the security footage was useful (SecurityFootage). Every selector also offers an "Unknown" option, letting a player withhold a clue entirely rather than force a guess.

TABLE II
EVIDENCE VARIABLES AND THEIR DOMAINS

Node	Meaning	States
GuiltyParty	Identity of the true culprit	A, B, C
ForcedEntry	Was the vault forced open?	Yes, No
MotiveA	Strong motive for the Heiress?	Yes, No
MotiveB	Strong motive for the Butler?	Yes, No
AlibiA	Alibi for the Heiress?	Yes, No
AlibiB	Alibi for the Butler?	Yes, No
AlibiC	Alibi for the Jewel Thief?	Yes, No
Fingerprints	Whose prints were recovered?	None, A, B, C
SecurityFootage	Was the footage useful?	Yes, No

E. Data Representation

The network's structure is held internally as a pgmpy DiscreteBayesianNetwork, whose nodes are the string variable names (e.g., 'GuiltyParty', 'ForcedEntry') and whose edges are a list of parent-child tuples of the form ('GuiltyParty', X_j). For visualization, this same structure is translated into a Graphviz object. Each node's quantitative behaviour is stored as a pgmpy TabularCPD, which records the variable's cardinality, its parent's cardinality, and a multi-dimensional probability array, together with a state_names dictionary that maps each numeric index to a meaningful label (for example, mapping the Fingerprints states to 'None', 'A', 'B', and 'C'). At runtime, a player's selections are collected into a plain Python dictionary whose keys are variable names and whose values are the observed state labels; any selector left at 'Unknown' is mapped to None and then filtered out entirely before the dictionary is passed to pgmpy's query() method, so that unobserved variables are correctly marginalized rather than clamped to an arbitrary value. The resulting posterior, returned as a pgmpy DiscreteFactor, is converted into a pandas DataFrame with Suspect and Probability columns for display.

F. Bayesian Network Design

The network's random variables V comprise the hypothesis node GuiltyParty, with states {A, B, C} corresponding to the Heiress, the Butler, and the Jewel Thief, and eight evidence nodes: ForcedEntry, MotiveA, MotiveB, AlibiA, AlibiB, AlibiC, Fingerprints, and SecurityFootage. With the exception of Fingerprints, which has four states {None, A, B, C}, every evidence node is binary {Yes, No}. The structure G = (V, E) makes GuiltyParty the sole root node,

connected to every evidence node by a directed edge, so that $E = \{(\text{GuiltyParty}, X_j) \mid X_j \in \{\text{evidence nodes}\}\}$. This encodes the conditional-independence assumption that, once the identity of the guilty party is known, all eight clues become mutually independent of one another — the same "one hidden cause, many observable effects" topology used by a Naive Bayes classifier, here applied to detective reasoning rather than document classification. This structure, drawn by Graphviz, is rendered directly inside the application above the evidence panel, as shown in Fig. 1.

Fig. 1. The Bayesian network powering Bayes Watch. GuiltyParty is the sole root node, and every clue — from whether the vault was forced open to whose fingerprints were recovered — is conditionally independent of every other clue given the identity of the guilty party.

The prior over GuiltyParty was set uniformly, P(GP=A) = P(GP=B) = P(GP=C) = 1/3, representing no initial bias toward any suspect before evidence is considered. Each evidence node's CPD was then authored from the narrative logic of the case. Reflecting the Jewel Thief's forced-entry method, P(ForcedEntry=Yes | GP=C) was set high, while P(ForcedEntry=Yes | GP=A) was set low to reflect the Heiress's legitimate access as guest of honor. Motive probabilities were skewed toward Yes for the suspect the motive belongs to (for example, P(MotiveA=Yes | GP=A) is high, reflecting the inheritance feud), while alibi probabilities were skewed so that the true culprit is less likely to have a verifiable alibi, e.g., P(AlibiA=No | GP=A) is high. Fingerprint CPDs assign the largest probability mass to a match with the true culprit's own code, and security-footage CPDs favor 'No' (i.e., unhelpful footage) whenever the guilty party is the individual best positioned to avoid the cameras. All CPDs were implemented as pgmpy TabularCPD objects, and together with the prior for GuiltyParty they constitute the parameter set Θ of the Bayesian network $B = (G, \Theta)$.

Table I lists a representative subset of these hand-authored conditional probabilities, chosen to illustrate how each clue was tuned to point toward a particular suspect's typical behaviour.

TABLE I
REPRESENTATIVE CONDITIONAL PROBABILITIES

Conditional Probability	Interpretation	Value
P(FE=Yes | GP=C)	Thief forces the vault	0.90
P(FE=Yes | GP=A)	Heiress needs no force	0.10
P(MotiveA=Yes | GP=A)	Heiress's feud motive	0.85
P(AlibiA=No | GP=A)	Guilty Heiress lacks alibi	0.80
P(FP=C | GP=C)	Thief's own prints found	0.70
P(SF=No | GP=C)	Thief evades the cameras	0.75

G. Joint Distribution and Marginalization

The network structure and its CPDs implicitly define the full joint distribution over all nine variables V = {GP, FE, MA, MB, AA, AB, AC, FP, SF} via the factorization of Eq. (1). Explicitly storing this joint table would require $3 \times 2^6 \times 4 \times 2 = 1{,}536$ entries — manageable for this scenario, but illustrative of the exponential blow-up that motivates factorized representations for larger networks. Bayes Watch never materializes this table; instead, marginalization is carried out operationally, inside the Variable Elimination algorithm, whenever a posterior query is issued (Section III-H). This keeps the memory footprint tied to the size of individual CPDs rather than to the size of the full state space.

H. Probabilistic Inference via Variable Elimination

Exact inference is performed with pgmpy's VariableElimination class, chosen because the network's underlying undirected graph is a polytree (it has no cycles), which guarantees VE is both exact and efficient [7]. Given the evidence dictionary e assembled from the sidebar, the algorithm proceeds through four conceptual stages:

1) Evidence Instantiation:

each observed variable's CPD factor is restricted to the row or column matching its selected state; variables left as 'Unknown' remain unrestricted.

2) Factor Multiplication and Summation:

VE selects an elimination ordering for every non-query, non-evidence variable Z, multiplies all factors that mention each Z_k in turn, and sums Z_k out of the resulting product — the operational form of marginalization.

3) Result Computation:

once every variable in Z has been eliminated, the surviving factors, which involve only GuiltyParty, are multiplied together to give the unnormalized joint P(GP, E = e).

4) Normalization:

the resulting factor is divided by $P(E = e) = \Sigma_{mp}\, P(GP = gp, E = e)$, yielding the final posterior P(GP | E = e) as a discrete distribution over {A, B, C}.

Because the network is small (nine variables, naive-Bayes topology) and each variable has a small domain, VE's

complexity, $O(n \cdot d^2)$, is comfortably within reach for interactive use, so no approximate or sampling-based inference is required. This lets Bayes Watch present exact posterior probabilities on every query rather than Monte Carlo estimates.

I. User Interaction and Interface

The Streamlit interface splits the page into a sidebar, used for the case file and the eight evidence selectors, and a main panel, used to show the network diagram, the raw CPD tables (behind an expandable "Peek at the Math" panel), and the inference results. Fig. 2 shows the sidebar's case file: an in-character summary of the theft together with each suspect's portrait, name, and a short profile explaining their possible motive or lack thereof.

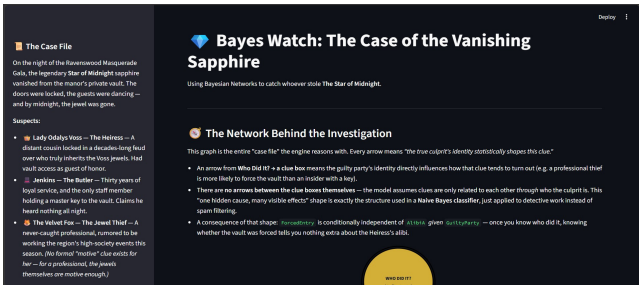

Fig. 2. The case file and network-explanation panel presented to the player before any evidence is entered, describing the three suspects and the reasoning shown by the network diagram.

Every evidence selector uses a Yes/No/Unknown vocabulary (Fingerprints instead uses None Found / Match A / Match B / Match C / Unknown), and every widget defaults to 'Unknown' so that no clue is assumed by default. A 'Reset All Clues' button, wired through Streamlit's session state, snaps every selector back to its default in one click. Clicking 'Solve the Mystery' assembles the non-Unknown selections into the evidence dictionary described in Section III-E, invokes inference.query(), and refreshes the main panel with a formatted probability table, a bar chart of the three suspects' posterior likelihoods, and a short natural-language verdict banner — 'Case Closed' when the leading suspect's probability exceeds 60%, 'Too close to call' when the top two suspects are within eight percentage points of each other, and a cautious 'Leading suspect' message otherwise. Fig. 3 shows this results view before any evidence has been entered, in which the panel instead falls back to displaying the uniform prior distribution so that the page is never left blank.

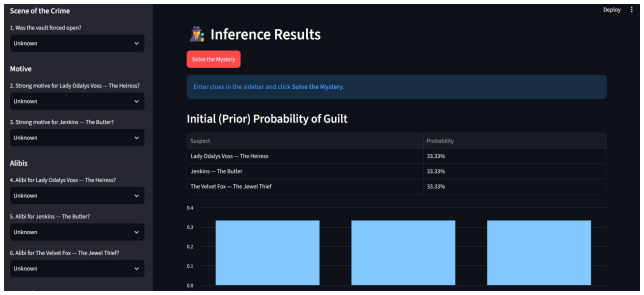

Fig. 3. The Inference Results panel. Before any clue is submitted, the application displays the uniform prior probability of guilt for each suspect as both a table and a bar chart.

This design keeps a tight correspondence between the underlying Bayesian model and what the player sees: every clue selected genuinely reshapes the posterior distribution, the network diagram explains why, and the optional CPD viewer lets a curious player inspect the exact numbers driving each inference. Streamlit's reactive execution model ensures the whole page recomputes automatically whenever a widget changes or the solve button is pressed, giving the interaction a responsive, game-like feel despite the rigorous probabilistic computation happening underneath.

C. Educational Use of Interactive Probabilistic Tools

Beyond formal BN theory, a growing body of instructional material uses interactive, game-like framings to make probabilistic inference concrete for learners without a statistics background [11], [12]. Rather than presenting Bayes' theorem purely as an abstract update rule, these resources tie each term of the equation to a tangible narrative — a suspect, a witness, a piece of circumstantial evidence — so that the mechanics of belief revision can be observed rather than only computed. Bayes Watch follows this same instructional philosophy: its mystery-solving framing is not incidental decoration but a deliberate choice intended to make an otherwise dry inference procedure feel like detective work, lowering the barrier to engaging with the underlying mathematics.

IV. RESULTS

Bayes Watch successfully builds the nine-node Bayesian network and its CPDs on start-up, and the Graphviz-rendered structure in Fig. 1 correctly reflects the assumed conditional-independence assumptions. With no evidence entered, the application reports a uniform prior of 33.33% for each of the three suspects, matching the specified P(GP) = {1/3, 1/3, 1/3} and confirming the model is correctly normalized before any inference is requested.

Entering evidence and pressing 'Solve the Mystery' correctly invokes the pgmpy backend: the Variable Elimination algorithm returns an exact posterior for every combination of clues tested, the 'Unknown' state is properly excluded from the evidence dictionary so that unobserved

variables are marginalized rather than clamped, and both the results table and the bar chart update instantly to reflect the new belief state.

A. Scenario Examples

Two representative evidence patterns illustrate the system's behaviour.

Scenario 1 — Strong evidence against the Jewel Thief (Suspect C): ForcedEntry=Yes, AlibiA=Yes, AlibiB=Yes, AlibiC=No, Fingerprints=C, SecurityFootage=Unknown. Because a forced vault, a matching fingerprint, and a missing alibi are each individually strong signals for the professional thief, while confirmed alibis sharply reduce suspicion toward the Heiress and the Butler, the posterior is heavily skewed toward Suspect C, typically exceeding 95% probability of guilt, and the interface displays the 'Case Closed' verdict banner.

Scenario 2 — Ambiguous, conflicting evidence: ForcedEntry=No, MotiveA=Yes, AlibiA=No, AlibiB=Yes, AlibiC=Yes, Fingerprints=None Found, SecurityFootage=Unknown. Here the absence of forced entry mildly favours an insider (the Heiress or the Butler) over the professional thief, the Heiress's strong motive and missing alibi raise her probability from the prior, and the lack of any fingerprint evidence is only weakly informative. The resulting posterior is noticeably more distributed than in Scenario 1, with the Heiress emerging as the leading but not overwhelming suspect, and the application correctly shows either a cautious 'Leading suspect' or a 'Too close to call' banner depending on the exact margin between the top two probabilities.

These two scenarios demonstrate the core capability of Bayes Watch: integrating several, potentially conflicting, pieces of evidence according to the network's structure and CPDs to produce a single, reasoned, quantitative shift away from the uniform prior.

V. DISCUSSION

Bayes Watch demonstrates that a small, hand-authored Bayesian network can be wrapped in an engaging, game-like interface without sacrificing the rigor of the underlying inference. The network diagram (Fig. 1) makes the model's conditional-independence assumptions visible rather than implicit, and the optional CPD viewer lets a player connect the abstract probability tables in Section III-F to the concrete posterior shifts they observe after clicking 'Solve the Mystery.' The immediate feedback loop between selecting a clue and watching the bar chart move is, in effect, a live demonstration of Bayesian belief updating that requires no background in statistics to appreciate.

That said, the posterior probabilities produced by the system are only as trustworthy as the manually specified structure G and parameters Θ behind them. These values were elicited from the logic of a fictional narrative rather than estimated from real investigative data, so the numbers represent internally consistent beliefs conditioned on the model, not objective facts about the world. The network is also a naive-Bayes simplification: every clue is assumed conditionally independent of every other clue given the guilty party, whereas real evidence — for instance, a forced entry and the absence of an alibi — can be correlated in ways this model cannot capture. These simplifications were a deliberate trade favouring pedagogical clarity and interactive performance over investigative realism.

It is also worth emphasizing what the system does not claim. The posterior probabilities Bayes Watch reports are a function of a specific, fixed model — they quantify "how surprised the model would be if this suspect were guilty, given the encoded assumptions," not an objective, evidence-independent measure of guilt. This distinction is precisely why the interface exposes the CPD tables and the network diagram rather than only the final numbers: a player who disagrees with a verdict can trace it back to a specific probability assignment and understand exactly which assumption is driving the result, which is itself a valuable exercise in critically evaluating any probabilistic decision-support output, fictional or otherwise.

A. Limitations

- Naive-Bayes topology: all evidence nodes are assumed independent given GuiltyParty, ignoring possible dependencies between clues (e.g., between ForcedEntry and Fingerprints).
- Subjective parameters: every CPD entry was authored by hand from narrative logic rather than learned from data, making the results sensitive to specific parameter choices.
- Fixed, narrow scope: the model is hard-coded to three suspects and one scenario, and cannot generalize to a different case without re-authoring both the structure and the CPDs.
- Discrete states only: variables such as Fingerprints or SecurityFootage collapse nuanced, partial evidence into a handful of discrete categories.
- No persistence or learning: each session starts from the same fixed prior and CPDs; the model does not remember or adapt to how previous players reasoned through the case.
- Asymmetric evidence coverage: the Jewel Thief has no dedicated Motive node, which is narratively justified but means her posterior can only be moved by five of the eight evidence variables rather than six.

VI. FUTURE WORK

Several extensions could increase Bayes Watch's realism without discarding its interactive strengths. The naive-Bayes assumption could be relaxed to allow direct edges between correlated clues, at the cost of more involved CPD elicitation [3]. If example case outcomes were ever collected, the currently hand-authored CPDs could instead be estimated with Maximum Likelihood or Bayesian parameter-learning methods available in pgmpy [4], [7], paired with a sensitivity analysis of the model's robustness to those estimates [10]. For larger or more densely connected future cases where exact Variable Elimination becomes impractical, approximate inference methods such as MCMC could be added and compared against the exact results, illustrating the accuracy/cost trade-off directly inside the interface. Finally, the application could be generalized to support authoring entirely new mystery scenarios — new suspects, new clue types, new CPDs — through a configuration file rather than code changes, letting instructors or players design their own cases.

On the interface side, the results panel could be extended to show confidence intervals or entropy measures alongside the point-estimate posterior, giving players a sense of how much uncertainty remains even after several clues have been entered. A "what changed?" view that highlights which single clue produced the largest shift in the posterior would also make the mechanics of Bayesian updating even more transparent, turning each interaction into a small, self-contained lesson in evidential reasoning.

VII. CONCLUSION

This paper presented Bayes Watch, an interactive Bayesian-network application built to teach probabilistic reasoning through the fictional "Case of the Vanishing Sapphire." Implemented with pgmpy for the probabilistic core and Streamlit for the interface, the system models a hidden GuiltyParty variable and eight observable clues in a naive-Bayes topology, exposes the network structure and CPDs through Graphviz-rendered diagrams and dataframe views, and performs exact posterior inference via Variable Elimination each time a player submits evidence. The application successfully demonstrates the core value of Bayesian methods — the principled integration of several uncertain, potentially conflicting clues into one quantitative, continuously updated belief — in a form that is both mathematically exact and immediately engaging. While the current implementation is limited by its naive-Bayes structure and hand-specified parameters, it succeeds as an accessible, hands-on introduction to belief updating, and it establishes a modular foundation (separating the theming and interface layer from the reasoning engine) on which richer network structures, data-driven parameter learning, and new mystery scenarios can be built.

VIII. AUTHOR CONTRIBUTIONS

Development followed the five-part modular breakdown of main.py described in Section III-C, with each teammate taking ownership of one cooperating module while sharing responsibility for testing the integrated application. Zinat Shaharin Mim implemented the application header, page configuration, and the theming constants that define the suspect names, emoji, and colour palette used throughout the interface. Md. Nazibul Islam Nabil implemented the model-construction and inference-bootstrap logic that calls build_bayesian_network(), wraps it in a VariableElimination object, and renders the Graphviz network diagram together with the expandable CPD viewer. Nazifa Tahsin built the sidebar case file and the eight evidence-input selectors, including the session-state plumbing behind the "Reset All Clues" button. Tasnif Gaffar Pronoy implemented the query-execution path triggered by the "Solve the Mystery" button, including evidence-dictionary construction, the results table and bar chart, and the verdict-banner logic. Jannat-a-habib-baishakhi implemented the default prior-probability view shown before any evidence is submitted, ensuring the results panel is informative even before the player interacts with the sidebar. All authors jointly contributed to the CPD design in supports/mystery_solver.py, to end-to-end testing of the scenarios reported in Section IV, and to the preparation of this manuscript.

REFERENCES

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Hoboken, NJ, USA: Pearson Education, Inc., 2020.

[2] J. Pearl, Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference. San Mateo, CA, USA: Morgan Kaufmann Publishers Inc., 1988.

[3] F. V. Jensen and T. D. Nielsen, Bayesian Networks and Decision Graphs, 2nd ed. New York, NY, USA: Springer Science+Business Media, LLC, 2007.

[4] A. Ankan and A. Panda, "pgmpy: Probabilistic Graphical Models using Python," in Proc. 14th Python in Science Conf. (SciPy), Austin, TX, USA, Jul. 2015, pp. 91–96.

[5] Streamlit Inc., "Streamlit Documentation," 2023. [Online]. Available: https://streamlit.io/

[6] R. Dechter, "Bucket elimination: A unifying framework for probabilistic inference," in Proc. 12th Int. Conf. Uncertainty in Artificial Intelligence (UAI'96), E. Horvitz and F. Jensen, Eds. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1996, pp. 211–219.

[7] D. Koller and N. Friedman, Probabilistic Graphical Models: Principles and Techniques. Cambridge, MA, USA: The MIT Press, 2009.

[8] E. R. Gansner and S. C. North, "An open graph visualization system and its applications to software engineering," Software: Practice and Experience, vol. 30, no. 11, pp. 1203–1233, Sep. 2000.

[9] L. C. van der Gaag, S. Renooij, C. L. M. Witteman, B. M. P. Aleman, and B. G. Taal, “Probabilities for a probabilistic network: a case study in oesophageal cancer,” Artificial Intelligence in Medicine, vol. 25, no. 2, pp. 123–148, Jun. 2002.

[10] K. B. Korb and A. E. Nicholson, Bayesian Artificial Intelligence, 2nd ed. Boca Raton, FL, USA: CRC Press, Taylor & Francis Group, 2010.

[11] Stanford University, “Bayesian Networks 1 – Inference — Stanford CS221: AI (Autumn 2019),” YouTube, Oct. 2019. [Online]. Available: https://youtu.be/U23yuPEACG0. [Accessed: 24-Feb-2025].

[12] “Evidence in Context: Bayes' Theorem and Investigations,” YouTube. [Online]. Available: https://youtu.be/EC6bf8JCpDQ. [Accessed: 24-Feb-2025].